AI Code Reviewer

Technical documentation

University project

October 2026

Contents

1 Overview **3**
- 1.1 What a review is . . . 3
- 1.2 The five parts . . . 3
- 1.3 What the demo shows . . . 4

2 Architecture **5**
- 2.1 Processes . . . 5
- 2.2 One language, shared types . . . 6
- 2.3 Whose keys are used . . . 6
- 2.4 Queues . . . 6

3 Data model **7**
- 3.1 Tables . . . 8
- 3.2 Embeddings . . . 8
- 3.3 What is deliberately not logged . . . 8

4 Review pipeline **9**
- 4.1 Accept the event . . . 9
- 4.2 Understand the change . . . 9
- 4.3 Ask the model, then rank . . . 10
- 4.4 The timeline . . . 10

5 Indexing **11**
- 5.1 What gets stored . . . 11
- 5.2 Incremental updates . . . 12
- 5.3 When indexing is unavailable . . . 12

6 API and MCP **13**
- 6.1 Routes . . . 13
- 6.2 MCP tools . . . 14
- 6.3 Slash commands . . . 14

7 Dashboard **15**
- 7.1 Live updates . . . 15
- 7.2 Landing . . . 15
- 7.3 Overview . . . 16
- 7.4 Repositories . . . 16
- 7.5 One repository . . . 17
- 7.6 Review detail . . . 18
- 7.7 Quality . . . 19
- 7.8 AI providers . . . 19
- 7.9 API keys . . . 19

8 Running and deploying **21**
8.1 Local stack 21
8.2 Vercel and Railway 22
8.3 What the demo does not include 22

Chapter 1

Overview

AI Code Reviewer reviews a code change with more than the diff. It parses the changed files, runs static rules, retrieves similar code from an index of the repository, and asks a language model for findings. Those findings are stored, shown in a dashboard, and, for a GitHub pull request, posted as one review with inline comments.

The same pipeline is available before a push. An MCP server exposes tools that Claude Code can call, including a review of a local `git diff`.

This is a university demo. The interesting part is the path from a pull-request event to a ranked set of comments, not multi-tenant scale. The stack runs locally with Docker Compose, or as a split deployment with the dashboard on Vercel and the API, worker, and database on Railway.

1.1 What a review is

A review is one run of the pipeline. It records who triggered it, which model answered, how many tokens it used, how long it took, a short summary, and a list of findings. Each finding has a file, a line range, a severity, a category, a message, and an optional suggestion.

Severity is one of critical, high, medium, low, or info. Category is one of bug, security, performance, style, or maintainability. A finding comes from the model or from Semgrep.

Two triggers matter in the demo:

- A GitHub pull request is opened, updated, or reopened on a repository that has reviews turned on. Draft pull requests are ignored.
- A developer sends a raw diff through the MCP tool `review_diff`.

A third trigger, manual re-run, repeats the review of a pull request from the dashboard. Eval runs use the same pipeline on a fixed set of diffs and do not post to GitHub.

1.2 The five parts

The repository is a pnpm workspace. Each part meets the others at the HTTP API and at the shared Zod schemas. Table 1.1 is the split of ownership.

Table 1.1: Modules and what each one owns.

Module	Owns
Git integration	GitHub App, webhooks, installation tokens, posting the review
Indexer	Clone, Tree-sitter chunking, embeddings, pgvector search
Review engine	Queue worker, prompts, Semgrep, validation and ranking, model calls
Dashboard	Next.js app, Clerk sign-in, repository and review screens
MCP and contracts	MCP tools and prompts, API keys, shared Zod schemas

The four apps live in `apps/`: web, api, worker, and mcp. Shared libraries live in `packages/`: db (Drizzle schema and migrations), shared (Zod schemas, queue names, job payloads), and llm (one interface over several model providers). `evals/` holds the sample diffs used to score the reviewer.

1.3 What the demo shows

A walkthrough of the system is: sign in, create the GitHub App from the dashboard, enable a repository, index it, open a pull request that contains a known bug, watch the pipeline timeline, read the inline comments on GitHub, and run the same reviewer on a local diff from Claude Code. Chapter 8 is the concrete runbook. The chapters in between describe how that walkthrough works.

Chapter 2

Architecture

Everything that reads or writes a review goes through the Core API. The dashboard, the MCP server, and the GitHub webhook are clients of that API. The webhook handler does not review code itself: it checks the signature, drops duplicates, enqueues a job, and returns. Indexing and reviewing run in BullMQ workers so a slow model call cannot block the HTTP response.

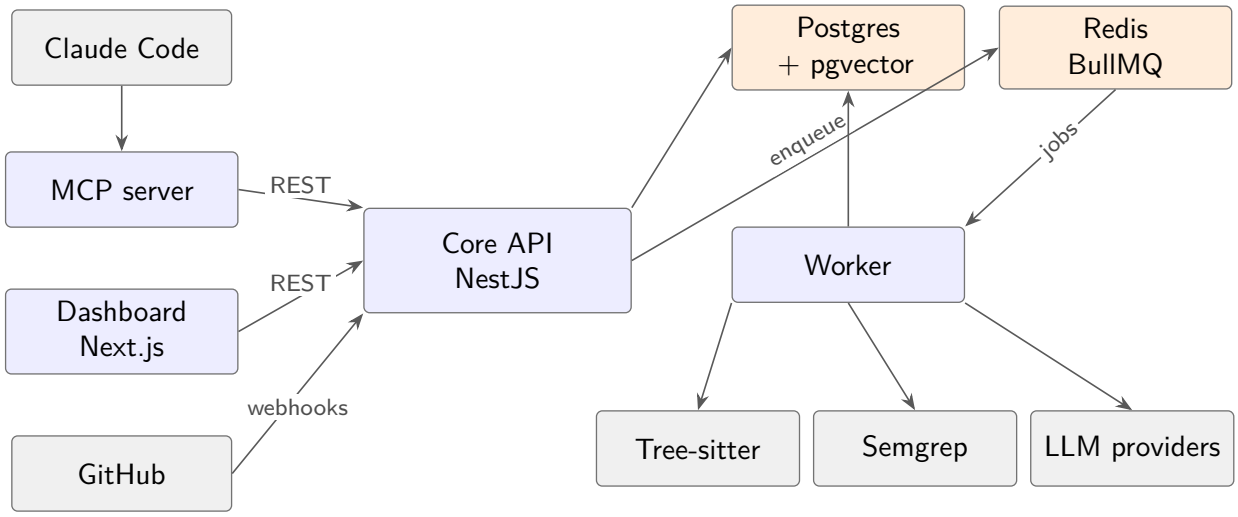


Figure 2.1: Runtime context. The worker posts the finished review back to GitHub. Postgres is the source of truth; Redis carries jobs and live events.

2.1 Processes

Dashboard. A Next.js application. Sign-in is Clerk. After sign-in the browser calls the API with the Clerk session.

Core API. A NestJS service. It owns REST routes, the webhook endpoint, API-key checks, and the server-sent event streams.

Worker. A Node process that consumes the `review`, `index-repo`, and `eval` queues. It is the only process that clones a repository, runs Semgrep, or calls a model during a review.

MCP server. A small process that speaks the Model Context Protocol over stdio or Streamable HTTP. It calls the Core API and does not open Postgres or GitHub itself.

Postgres. Relational data and the embedding index (pgvector) in one database.

Redis. BullMQ queues, rate limiting, and a pub/sub channel for live dashboard updates.

Tree-sitter and Semgrep are tools the worker shells out to or loads in process. Tree-sitter is the WebAssembly build (`web-tree-sitter`), so the worker does not need a native compiler. Semgrep is the Semgrep CLI, installed in the worker image. If Semgrep is missing, that stage is skipped and the review continues.

2.2 One language, shared types

The apps and packages are TypeScript, compiled as ES modules. A relative import in a package uses a `.js` suffix because the packages emit `dist/` with `tsc`, and Node resolves that suffix at runtime. Change a package, then build it, before an app will see the change. `pnpm dev` builds packages first.

Request and response bodies are Zod schemas in `packages/shared`. The API validates with those schemas. The dashboard and the MCP server import the same types. A new field is added once.

Model calls go through `packages/llm`. Business code does not import a provider SDK, except that the Claude implementation uses the official Anthropic SDK inside that package. The call is `generateJson`: a system prompt, a user prompt, and a Zod schema. The result is checked against the schema. One retry appends the validation error to the prompt. HTTP 429 and 5xx responses back off. If the primary provider fails, the configured fallback provider is tried. Calls are spaced out because the free tiers used in the demo are small.

The provider and model names come from configuration: `LLM_PROVIDER` and `LLM_MODEL`, with the same pair for the fallback. Accepted providers are Gemini, OpenRouter, any OpenAI-compatible host (NVIDIA NIM or Groq, for example), Anthropic, and OpenAI's own API. Embeddings are Gemini or OpenAI. Claude has no embedding model.

2.3 Whose keys are used

Each signed-in user has a row of provider settings. A job runs with the keys of the user it belongs to: the owner of the GitHub installation, or the person who requested an MCP review or an eval run. Keys stored for a user stay in the database and are not sent back to the browser.

Server environment keys are not shared by default. Only the GitHub App admin may use the keys from the environment. `SHARED_LLM_KEYS=true` shares them with every signed-in user, which is appropriate for a trusted team demo and not for a public deploy.

2.4 Queues

Three queues, named in `packages/shared`:

- `review` carries one review that already has a row in status `queued`.
- `index-repo` carries a full index or an incremental update of one repository.
- `eval` carries one run of the sample set.

Jobs retry with backoff. The webhook's idempotency check (Chapter 4) stops a GitHub retry from enqueueing a second review for the same delivery.

Chapter 3

Data model

Postgres holds users, GitHub installations, repositories, reviews, findings, and the code index. Figure 3.1 is the main graph. An arrow points from a row to the row it references. `reviews` also references `repositories` and `users`; those two links are omitted from the drawing so the pull-request chain stays readable. `webhook_deliveries` has no foreign key. Eval tables (`eval_runs`, `eval_case_results`) hang off `users` and are omitted from the figure for the same reason.

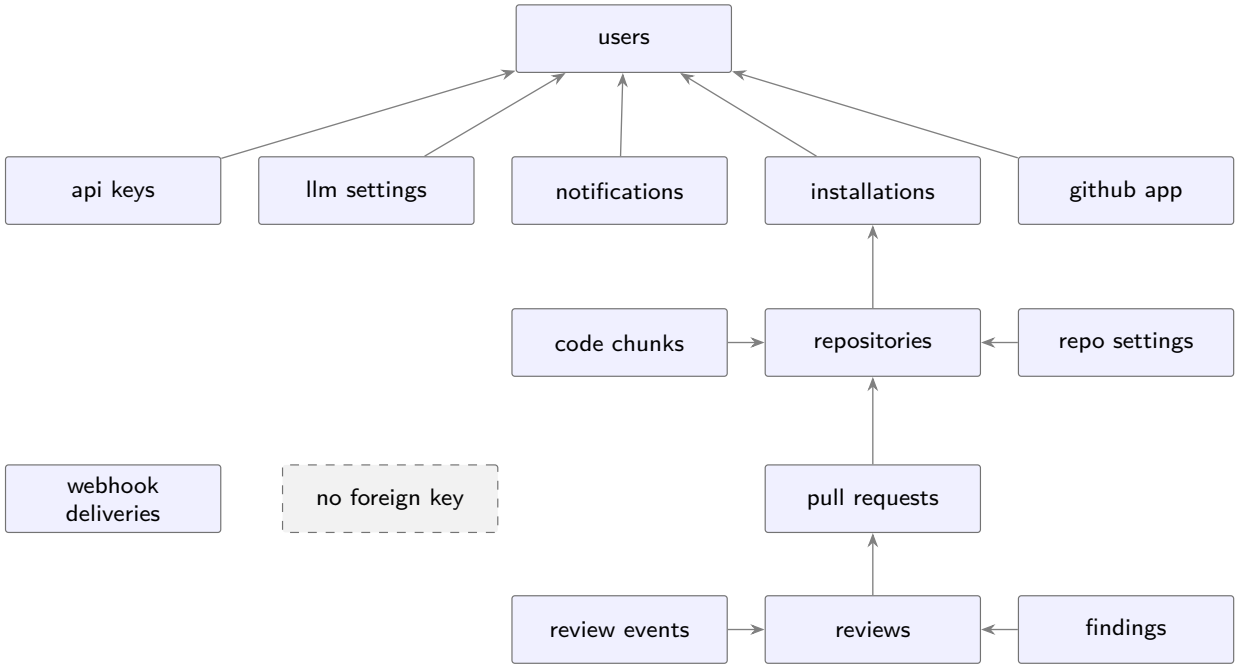


Figure 3.1: Primary relationships. `webhook deliveries` records a GitHub delivery id so a retry is not processed twice.

3.1 Tables

Table 3.1: What each table is for.

Table	Role
`users`	Clerk user id and email.
`installations`	A GitHub App installation. `user_id` is null until Connect GitHub finishes, because the webhook can arrive first.
`repositories`	One connected repo: enabled flag, index status, last indexed commit, and the embedding model the index was built with.
`repo_settings`	Strictness, ignored paths, custom rules, and the maximum number of comments.
`pull_requests`	Number, title, author, head and base commits, and open, closed, or merged. Unique per repository and number.
`reviews`	One pipeline run. Pull request and repository are null for a local MCP review; `user_id` is who asked. Stores provider, model, token counts, duration, summary, and error.
`findings`	One comment: file, lines, severity, category, source (`llm` or `semgrep`), message, suggestion, confidence, and whether it was posted.
`review_events`	One row per pipeline stage, which is the timeline on the review page.
`code_chunks`	A symbol or a line window, its text, a content hash, and a vector. An HNSW index serves cosine search.
`api_keys`	A prefix and a hash. The raw key is returned once at creation and is not stored.
`llm_settings`	One row per user: provider keys and model choices. A null column means "use the server default", subject to the admin rule in Chapter 2.
`github_app`	The single App created from the dashboard (id, slug, private key, webhook secret, and the admin user). Environment variables are only a fallback.
`notifications`	"Review finished", "indexing failed", and the other worker messages, with an optional dashboard link.
`webhook_deliveries`	Primary key is the GitHub delivery id.

3.2 Embeddings

The vector column width is the `EMBEDDING_DIM` used at migration time, 768 by default. It has to match the model. `repositories.embedding_model` stores the `provider:model` pair that built the index. Indexing with a different embedder deletes the old chunks and rebuilds. Search and review context refuse an index that was built with another embedder, and the dashboard asks for a re-index.

Chunks are unique by content hash, so a later index does not re-embed a function that did not change.

3.3 What is deliberately not logged

The worker writes short stage messages to `review_events`. Those messages do not include source code. The chunk table does store chunk text, because search and review context need it. The clone used during indexing is deleted when the job ends. API keys are stored as hashes. GitHub installation tokens are short-lived and are not written to a table.

Chapter 4

Review pipeline

A review starts from a GitHub `pull_request` action (`opened`, `synchronize`, `reopened`), from a manual re-run, or from a raw diff. The webhook path is the one drawn in Figure 4.1. The worker path is the same after the diff is in hand.

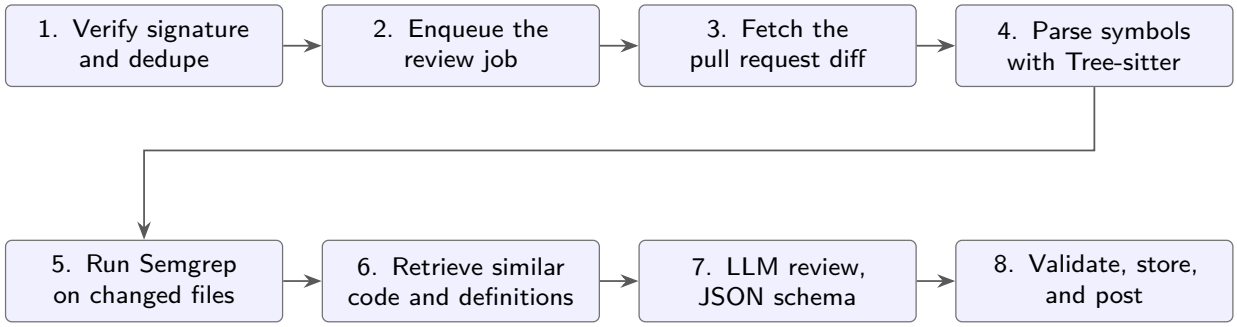


Figure 4.1: From a GitHub delivery to a posted review. Parse, Semgrep, retrieval, and the model call run in the worker.

4.1 Accept the event

The endpoint is `POST /webhooks/github`. It checks `X-Hub-Signature-256` against the App webhook secret. It inserts `X-GitHub-Delivery` into `webhook_deliveries`. A conflict means GitHub retried a delivery the API already accepted, and the handler returns without enqueueing again. The handler then returns as soon as the job is on the queue. The target is a response in under three seconds.

The repository must be enabled. A draft pull request is ignored. A push to the default branch of an indexed repository is not a review: it enqueues an incremental index, described in Chapter 5.

On the pull request the worker adds an eyes reaction and opens an "AI Code Review" check run when the App has permission to do so. Both are best effort. The check run id is stored on the review row.

4.2 Understand the change

The worker takes an installation token and downloads the diff and the changed file contents. It skips lockfiles, generated files, binaries, and paths that match the repository's ignored globs.

Tree-sitter then finds the functions and classes that changed and the calls they make. Grammars are loaded for TypeScript, JavaScript, Python, Go, Java, Ruby, and Rust. Any other language is split into overlapping line windows. Definitions of the called symbols are loaded from `code_chunks` when the repository has been indexed with the current embedding model.

Semgrep runs on the changed files and emits findings with the same shape as the model. A missing Semgrep binary skips the stage. It does not fail the review.

4.3 Ask the model, then rank

For each changed file, or a group of hunks, the worker sends one prompt: the hunks, the top retrieved chunks, the called definitions, and the repository's custom rules. The prompt demands JSON. Each object has `file`, `line_start`, `line_end`, `severity`, `category`, `message`, `suggestion`, and `confidence`. Zod checks the array. A bad answer is retried once with the parser error included.

Ranking then does four things:

1. Drop a finding whose lines are not in the diff.
2. Merge duplicates that came from both the model and Semgrep.
3. Drop low-confidence notes and pure nitpicks according to strictness (`low`, `medium`, or `high`).
4. Sort by severity and keep at most `max_comments` findings. The default cap is 15, and the settings page allows 1 to 50.

The review row is updated with the summary, the provider, the model, token counts, and the duration. Findings are inserted. The worker posts a single GitHub review: a summary body plus one inline comment per surviving finding, then completes the check run.

4.4 The timeline

Each stage writes a `review_events` row: `fetch`, `static_analysis`, `context`, `llm`, `validate`, or `post`, with status running, done, skipped, or failed. That is the timeline on the review page. The worker also publishes a Redis event so the open page updates without a reload (Chapter 7).

A failed job retries. When retries are exhausted the review status becomes `failed`, the error is stored, and the repository owner is notified. A successful review notifies the same way. Notification text is a title and a short body, not the diff.

Chapter 5

Indexing

Indexing is a separate job from review. A review can run without an index; it then sees the diff, Semgrep, and the file contents, and it does not see the rest of the repository. With an index, step 6 of the pipeline can pull similar chunks and the definitions the change calls.

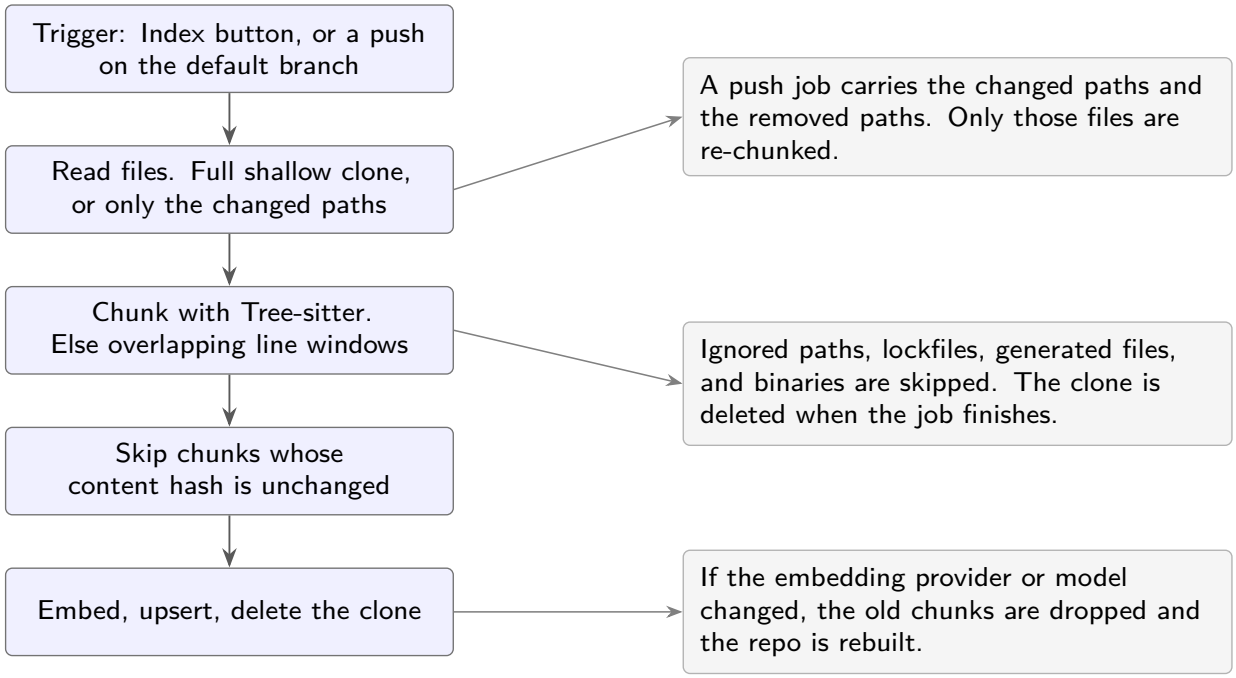


Figure 5.1: Full and incremental indexing share the chunk, hash, and embed steps.

5.1 What gets stored

The worker shallow-clones the default branch, walks the tree, and skips the same kinds of paths a review skips. Tree-sitter splits a supported file by symbol. A file it cannot parse becomes fixed-size windows with overlap. Each chunk stores the path, the symbol name, the language, the line range, the text, a content hash, the commit, and an embedding.

Unchanged hashes are not sent to the embedding API. New and edited chunks are embedded and upserted. Removed files drop their chunks. The clone directory is deleted before the job returns. Index status on the repository moves through `none`, `indexing`, `ready`, and `failed`, and a progress string is shown in the dashboard while the job runs.

5.2 Incremental updates

A `push` webhook on the default branch of a repository that already has an index enqueues `index-repo` with the list of changed paths and the list of removed paths. Only those files are re-chunked. The first index, and any index after an embedding-model change, walks the whole tree.

Search embeds the query with the caller's embedding provider and runs a cosine search over that repository's chunks. If the stored `embedding_model` does not match the caller's provider and model, search refuses and the dashboard offers a re-index. Mixing vectors from two models in one index would rank nonsense.

5.3 When indexing is unavailable

The Index button stays disabled until the user has an embedding provider configured: Gemini, or OpenAI. A review model such as Claude does not satisfy that requirement. Reviews still run. They have less context.

Chapter 6

API and MCP

The dashboard sends the Clerk session. The MCP server sends `Authorization: Bearer` with a key created on the API keys page. The webhook uses the GitHub signature and no session. Keys are stored as a hash; the `crk_` prefix is the only part shown again in the UI.

6.1 Routes

Paths below are the contract the dashboard and the MCP server share. `:id` is a repository or a review, depending on the prefix.

Table 6.1: Core API routes.

Route	Role
`GET /api/repos`	Connected repositories.
`POST /api/repos/:id/enable`	Turn reviews on or off.
`POST /api/repos/:id/index`	Enqueue a full index.
`GET` and `PUT /api/repos/:id/settings`	Strictness, ignored paths, custom rules, comment cap.
`GET /api/repos/:id/reviews`	Paginated review history.
`GET /api/reviews/:id`	One review and its findings.
`GET /api/reviews/:id/status`	`queued`, `running`, `completed`, or `failed`.
`GET /api/reviews/:id/events`	Pipeline timeline.
`POST /api/reviews/:id/rerun`	Review the pull request again on its latest known commit. One re-run at a time per pull request.
`POST /api/reviews/diff`	Start a review of a raw diff. Returns a review id.
`GET /api/stats`	Totals for the overview cards.
`POST /api/repos/:id/search`	Semantic search over chunks.
`GET /api/repos/:id/rules`	Effective review rules.
`GET /api/repos/:id/prs/:number/findings`	Latest findings for a pull request.
`GET /api/findings/:id`	One finding, used by the MCP explain tool.
`GET /api/github/install-url`	Where the Connect GitHub button sends the browser.
`GET /api/github/callback`	Saves the installation after GitHub redirects back.
`POST /webhooks/github`	Signature-checked webhook. No session.
`GET`, `POST /api/keys`, `DELETE /api/keys/:id`	Create, list, and revoke MCP keys. The raw key is in the create response only.
`GET /api/notifications/stream`	Server-sent events for the signed-in user.

Route	Role
`GET /api/reviews/:id/stream`	Server-sent events for one review.

Creating the GitHub App is a separate setup flow: the dashboard posts a manifest, GitHub redirects back, and the API stores the App credentials in `github_app`. Only the admin who created the App may replace it.

6.2 MCP tools

The server name is `codereview`. Tool results are compact JSON: file, line, severity, message, suggestion. They do not include whole files. `review_diff` posts the diff, then polls the review status until it finishes or about two minutes pass.

Table 6.2: Tools the agent can call.

Tool	What it does
`review_diff`	Review a unified diff. Optional `repo` (`owner/name`) adds index context and that repo's rules.
`get_pr_review`	Findings already produced for a pull request.
`search_codebase`	Semantic search. Needs an index.
`get_review_rules`	Strictness, custom rules, ignored paths.
`explain_finding`	One finding in full, by id.
`list_repos`	Connected repos, with enabled and index status.

6.3 Slash commands

MCP prompts show up in Claude Code as slash commands. With the server registered as `codereview` they are:

- `/mcp__codereview__review` runs `git diff`, calls `review_diff`, and summarizes by severity. It does not edit files.
- `/mcp__codereview__review_pr` takes a pull-request number and lists `get_pr_review`.
- `/mcp__codereview__fix_findings` loads those findings and fixes them one at a time in the working tree, then shows the diff. It does not commit.
- `/mcp__codereview__search` takes a query and explains `search_codebase`.

Stdio is the local demo: the API key is the environment variable `CODEREVIEW_API_KEY`, and logs go to stderr so they are not mixed into the protocol stream. Streamable HTTP is the remote form, with the same bearer key.

Chapter 7

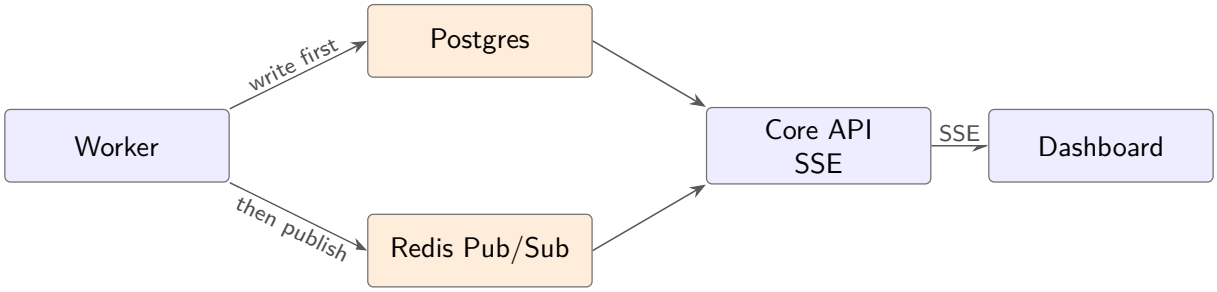
Dashboard

The dashboard is the Next.js app. Sign-in is required. The first time the app has no Clerk keys, the sign-in page asks for them and stores them for the API and the web process; keys in the environment take priority. After sign-in the sidebar is the same on every page: Overview, Repositories, Quality, AI providers, and API keys and MCP. A bell in the header lists notifications.

The wireframes in this chapter are drawings of those screens. They show structure, not the visual design of the running app.

7.1 Live updates

The worker writes Postgres first, then publishes on Redis. The API keeps one subscriber and fans events out to browsers.



Channels: `notify:user:<userId>` for the notification bell, and `review:<reviewId>` for the pipeline timeline. On each reconnect the dashboard reads the database again, and it polls slowly while the stream is down.

Figure 7.1: Live updates. The browser uses `fetch` for the stream because `EventSource` cannot send the Clerk `Authorization` header.

`GET /api/notifications/stream` feeds the bell. `GET /api/reviews/:id/stream` feeds the timeline of an open review. The stream sends a ready event on connect and a heartbeat every 25 seconds. When the last listener leaves, the API unsubscribes from that channel. If the stream drops, the page polls slowly and catches up from the database on the next connect.

7.2 Landing

The public page is a dark marketing screen: a short pitch, a link into the dashboard, and the three-step story of connecting a repository, opening a pull request, and receiving inline comments.

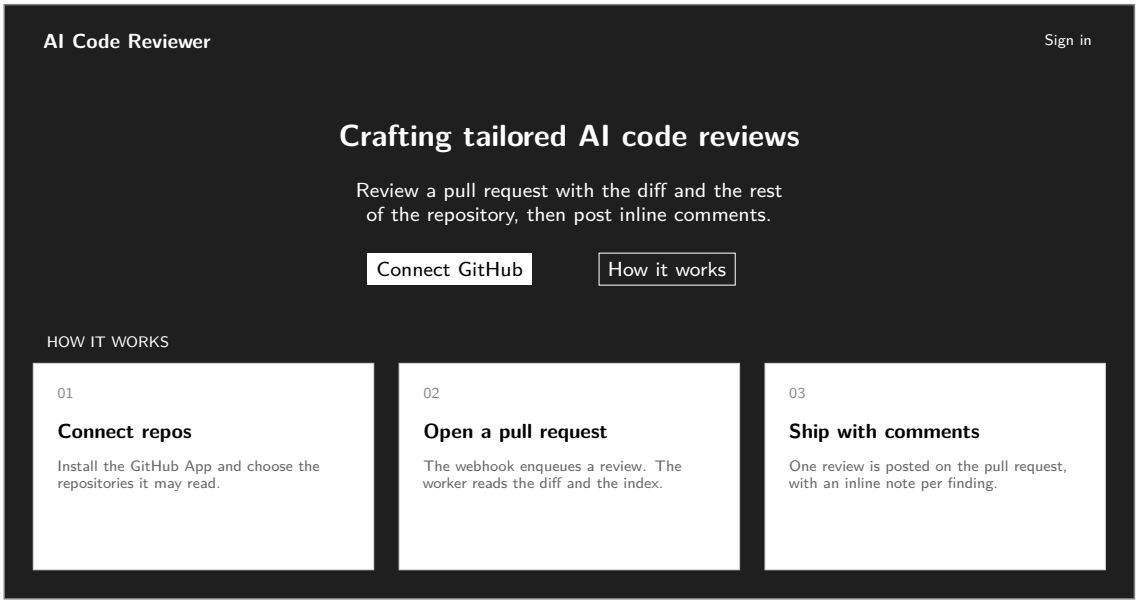


Figure 7.2: Landing page. Connect GitHub leads to sign-in and then the dashboard.

7.3 Overview

Three cards read `GET /api/stats`: review count, finding count, and how many repositories have reviews enabled. Under them, the latest reviews are merged from each repository, because there is no single "all reviews" route.

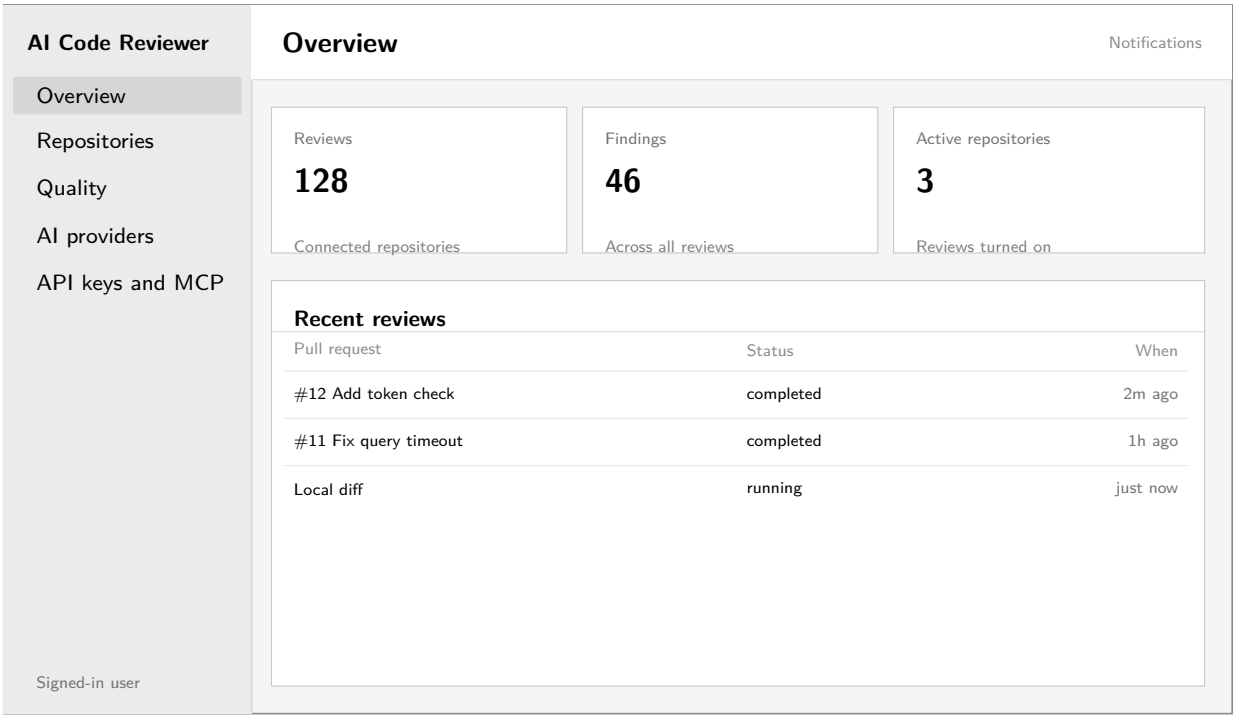


Figure 7.3: Overview. A row that is still queued or running is refreshed until it finishes.

7.4 Repositories

The list is every repository the GitHub App is installed on. The switch turns reviews on or off. Index starts a job and is disabled until an embedding provider is configured. Opening a row goes

to that repository.

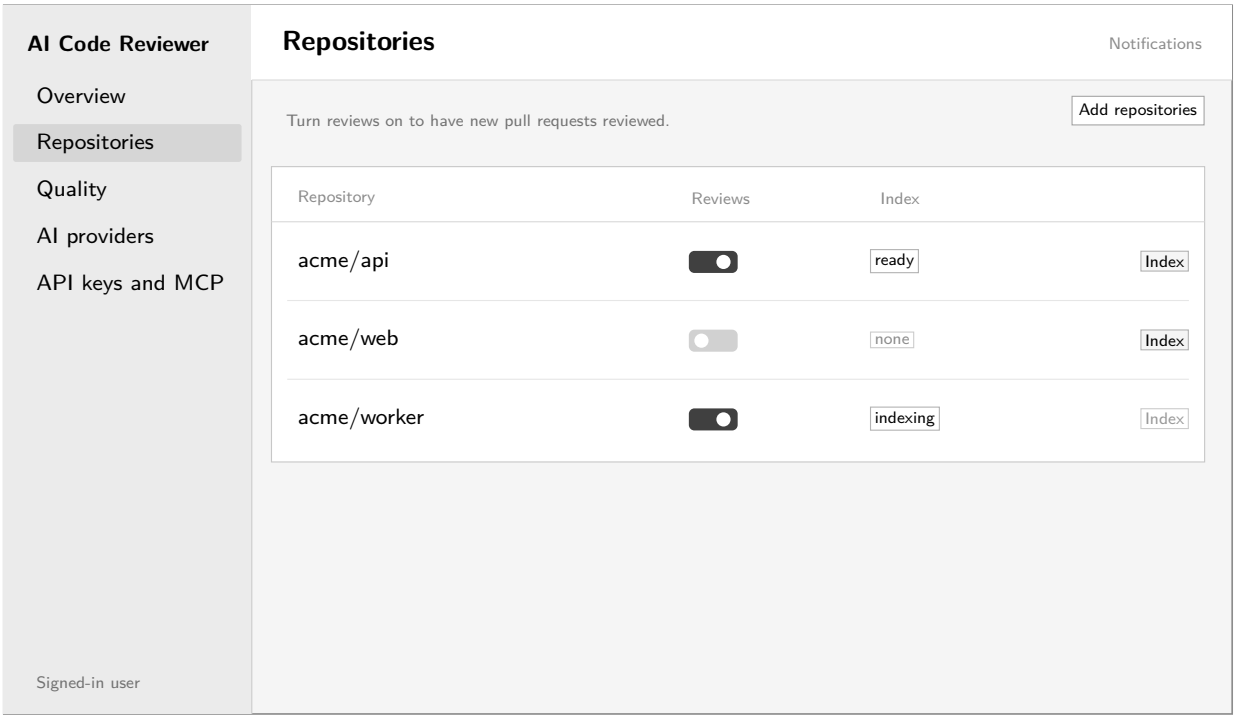


Figure 7.4: Repositories. Reviews off means a new pull request is not enqueued.

7.5 One repository

The repository page has two tabs. Reviews is the history for that repo, with a control to re-run a finished pull-request review. Settings edits strictness, the comment cap, ignored globs (one per line), and custom rules. Each custom rule is a sentence plus a severity. Saved settings apply to the next review, not the one already running.

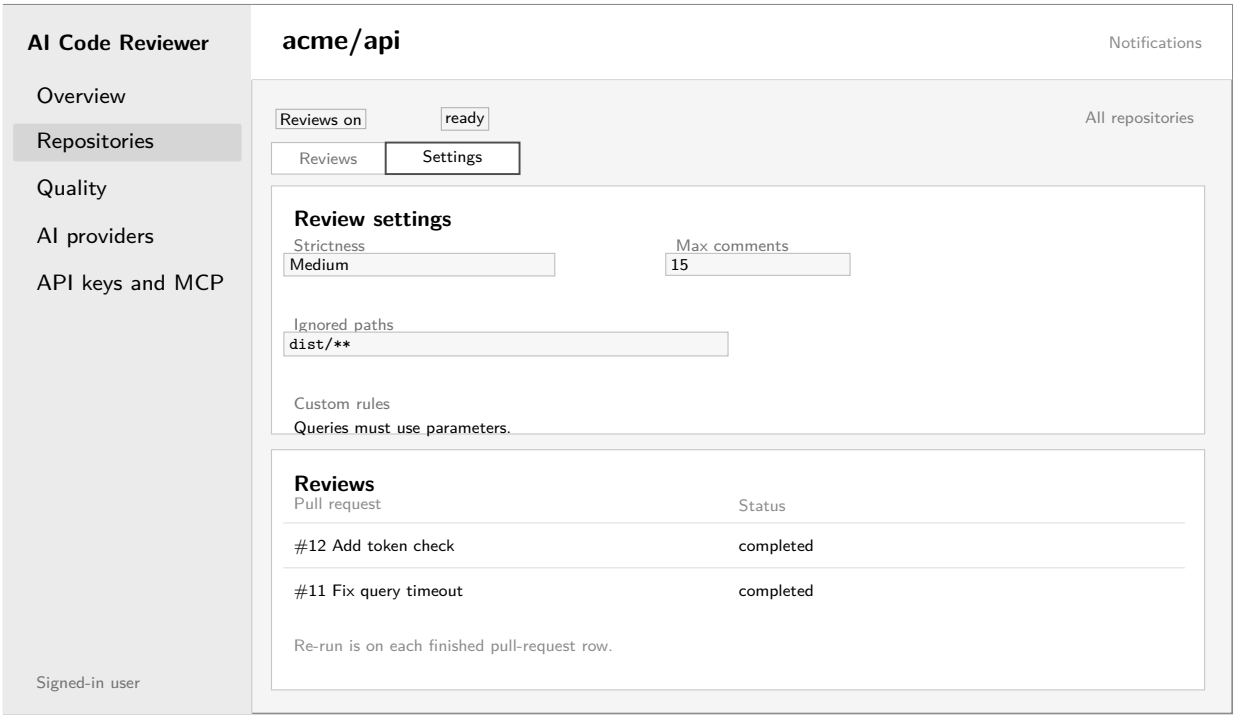


Figure 7.5: Repository page. The running app shows one tab at a time. Both are drawn here.

7.6 Review detail

The title is the pull-request number and title, or "Local diff review" when there is no pull request. The timeline is the `review_events` rows. Findings are grouped by file and can be filtered by severity. A finding shows its severity, category, lines, message, and the suggestion when there is one. Re-run is available once the review is no longer queued or running, and only for a pull request.

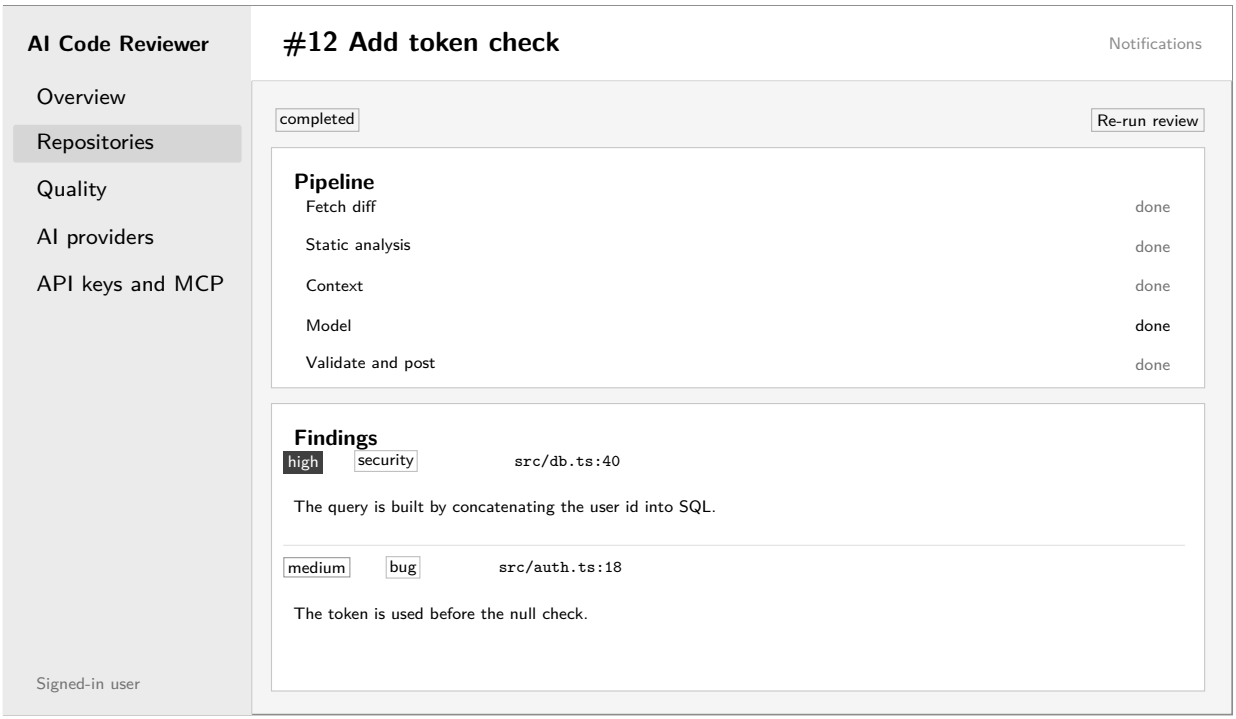


Figure 7.6: Review detail. The timeline advances as stage events arrive on the review stream.

7.7 Quality

Quality runs the diffs in `evals/` through the same pipeline and scores them: recall (planted issues that were found), precision (findings that hit a planted issue), and false alarms on the clean sample. The page charts those totals and lists each case. Nothing is posted to GitHub.

7.8 AI providers

The page shows which provider will actually run: the main reviewer, the fallback, and the embedding model used for indexing. Keys are split into a free tier and a paid tier, with a link to obtain each key. Saving a key stores it on the user's `llm_settings` row. Clearing the field falls back to the environment value when this user is allowed to use it.

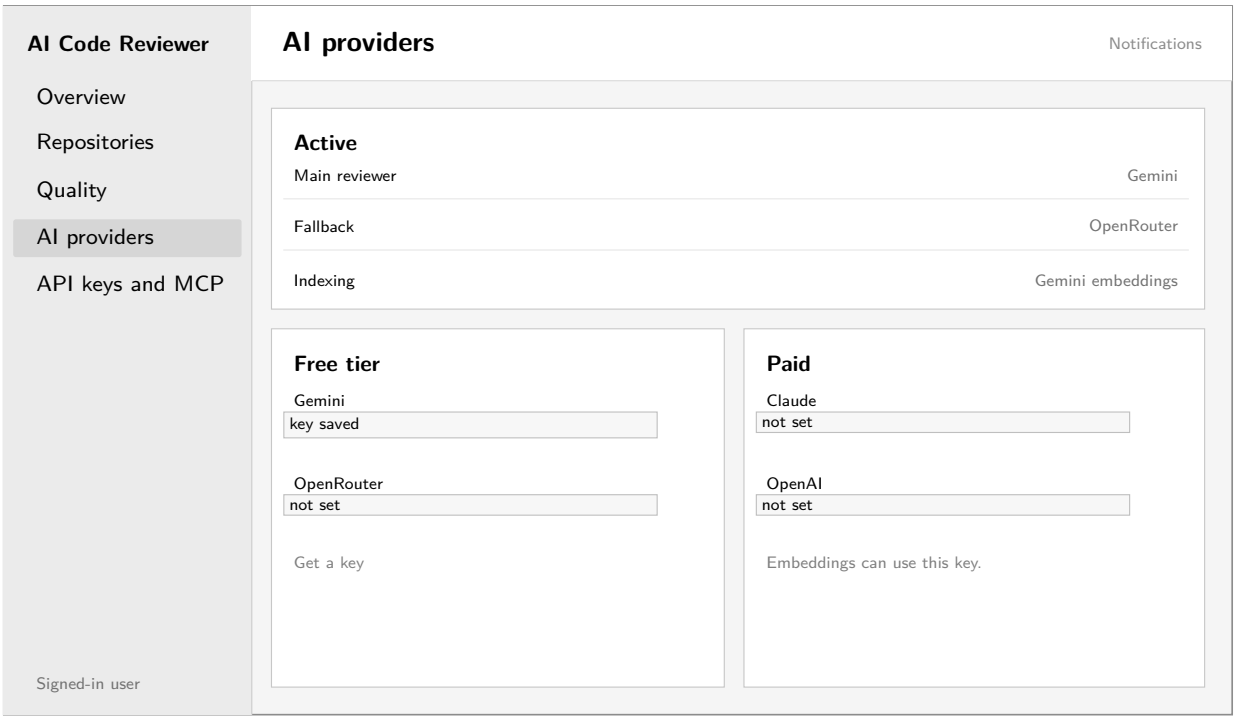


Figure 7.7: AI providers. Indexing stays unavailable until the embeddings row is configured.

7.9 API keys

Creating a key asks for a name, shows the raw `crk_` value once, and then lists only the prefix, the name, and the last time it was used. The page also shows the command that registers the HTTP MCP server with Claude Code.

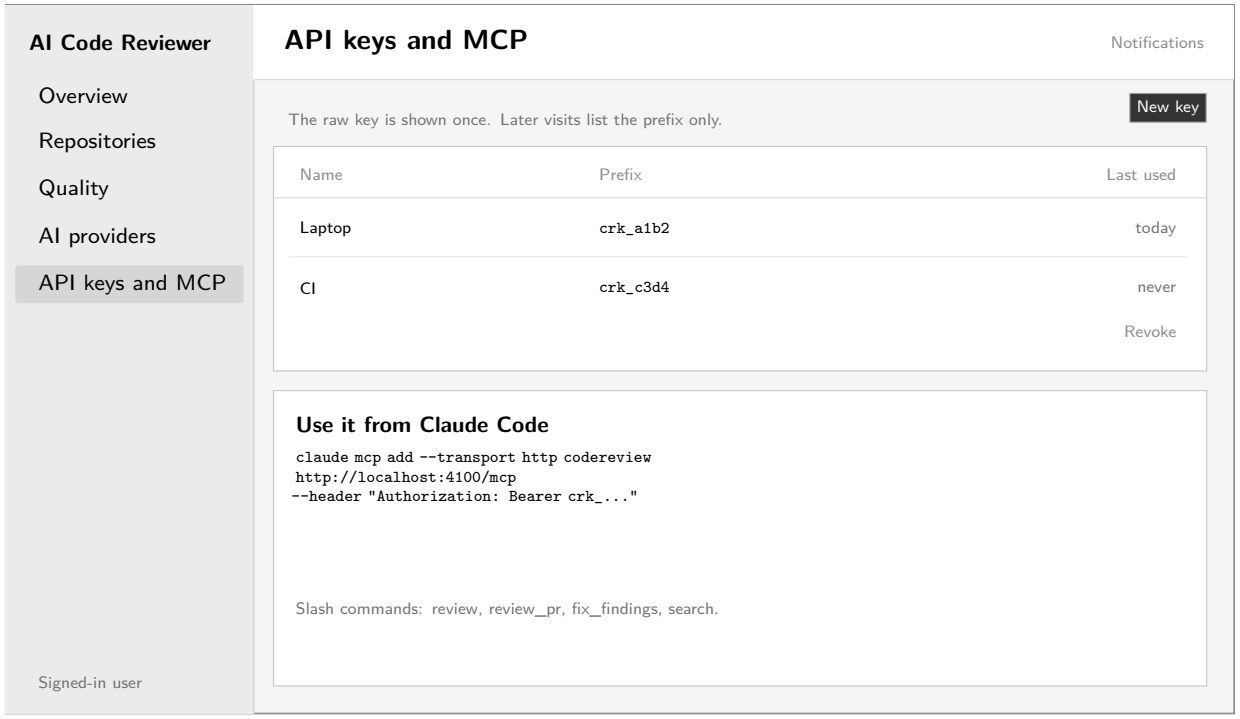


Figure 7.8: API keys. Revoking a key sets `revoked_at`. The hash cannot be turned back into the secret.

Chapter 8

Running and deploying

Two ways to run the system are supported: one Docker Compose stack on a laptop, or the dashboard on Vercel and everything else on Railway. Neither path puts secrets in the repository. Keys live in the environment, in the Clerk setup file on first run, or in `llm_settings` after a user pastes them.

8.1 Local stack

Docker Compose builds one image per service from the root Dockerfile. Migrations run before the API serves traffic. The worker image includes git and Semgrep.

Table 8.1: Containers in the local stack.

Container	Role
`postgres`	PostgreSQL 16 with pgvector
`redis`	Queues and pub/sub
`migrate`	Applies Drizzle migrations, then exits
`api`	Core API and the webhook, port 4000
`worker`	Review, index, and eval jobs
`web`	Dashboard, port 3000
`mcp`	MCP over HTTP, port 4100
`smee`	Forwards GitHub webhooks to the API through a public channel

```
docker compose up --build
```

Open the dashboard at `http://localhost:3000`. The first-run screen asks for the two Clerk keys if they are not already in the environment. Then:

1. Create the GitHub App from the dashboard. The compose stack has already created a public webhook URL, and the manifest flow points the App at it.
2. Choose which repositories to install the App on.
3. On Repositories, turn reviews on. Index the repo if an embedding key is configured.
4. Open a pull request. The bot reacts, starts the check run, and posts comments. The review page shows each stage as it finishes.

A second free provider is worth configuring. Gemini's free tier is small. An OpenAI-compatible host can be the fallback so a quota error does not fail the review.

To develop the TypeScript itself, start only Postgres, Redis, and the webhook forwarder in Docker, run migrations, then `pnpm dev` for the four apps. Point the forwarder at the API on the host, and set the webhook URL to the smee channel.

`pnpm eval` sends the sample diffs to a running API and prints recall, precision, and false alarms. The same action exists as Run on the Quality page.

8.2 Vercel and Railway

Production does not use the smee container. GitHub calls the API's public URL directly.

- Railway runs Postgres with pgvector, Redis, the API, the worker, and optionally the MCP server. The image target is selected with the `SERVICE` variable (`api`, `worker`, or `mcp`).
- Vercel runs `apps/web`. It needs the public API URL, the MCP URL if used, and the Clerk keys. The Vercel domain is added to the Clerk allow list.
- Create a new GitHub App from the deployed dashboard so its webhook URL is the Railway API, not the local smee channel.

The API's `WEB_URL` is the dashboard origin, used for redirects and for CORS. After the domains exist, set that variable and the public API URL, then create the GitHub App.

8.3 What the demo does not include

There is no billing, no GitLab or Bitbucket, and no fine-tuned model. Langfuse and Sentry are not wired up. Provider keys saved from the dashboard are stored as plain text, same as the GitHub App private key; that is an accepted demo limitation, called out in the schema. Do not point the demo at a private repository you cannot share with the model provider: free tiers may train on prompts.